

Advanced Lesson 8 - Exercises: Pattern Matching and Conditional Definitions

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

1: Why pattern matching belongs in functional arguments

Part 1. A definition written without a pattern, $f[x] = x^2$, attaches to the single symbol x and to nothing else. It is not a function; it is a stored value for one particular expression. Writing $f[x_] = x^2$ attaches to the pattern "any one argument", which is what a function of one variable actually means. Pattern matching is how a definition says which calls it applies to, and without it a definition applies to exactly one call.

```
Clear[g, x, y];  
g[x] = Sin[x];  
{g[x], g[y], g[0], g[ $\frac{\pi}{2}$ ]}
```



```
Clear[g, x, y];  
g[x_] = Sin[x];  
{g[x], g[y], g[0], g[ $\frac{\pi}{2}$ ]}
```

2: Why delayed assignment belongs in functional definitions

Part 1. Immediate assignment evaluates the right-hand side once, at definition time, when the pattern variable is still a bare symbol. Any

right-hand side whose meaning depends on the argument having a value - a derivative, an integral, a random draw, a conditional test, a loop - is therefore computed against the wrong thing and frozen. Delayed assignment stores the right-hand side unevaluated and re-evaluates it per call, which is what a function body is supposed to do.

```
Clear[h1, h2, x];
h1[x_] = Length[x];
h2[x_] := Length[x];
{h1[{1, 2, 3}], h2[{1, 2, 3}]}
```

3: Squares of integers, cubes of rationals, everything else unchanged

```
Clear[f, x, values];
f[x_Integer] := x2;
f[x_Rational] := x3;
f[x_] := x
??f
{f[ $\frac{3}{5}$ ], f[3], f[ $\frac{4}{2}$ ], f[3.1`]}
values = Table[{x, f[x]}, {x, -5, 5,  $\frac{1}{2}$ }]
ListPlot[values]
```

4: NumberQ against NumericQ

```
Clear[vals];
vals = {3,  $\frac{3}{4}$ , 3.1`, 3 + 4 i,  $\sqrt{2}$ ,  $\pi$ , e};
NumberQ /@ vals
NumericQ /@ vals
```

5: VectorQ against ListQ against MatrixQ

```
Clear[objs];
objs = {3, {1, 2, 3}, {{1, 2}, {3, 4}},
  {{1}, {2}}, {{3, 4}}, {{1, 2}, {3, 4}}, {{5, 6}, {7, 8}}};
ListQ /@ objs
```

```
VectorQ /@ objs
```

```
MatrixQ /@ objs
```

6: A sign-switched sine, via Boolean tests

```
Clear[f, x];
f[x_] := Sin[x] /; Positive[x];
f[x_] := -Sin[x] /; ! Positive[x]
Plot[f[x], {x, - $\pi$ ,  $\pi$ }]
```

7: Exercise 3 again, using conditional evaluation

```
Clear[f, x];
f[x_] := x2 /; IntegerQ[x];
f[x_] := x3 /; Head[x] === Rational;
f[x_] := x
{f[ $\frac{3}{5}$ ], f[3], f[ $\frac{4}{2}$ ], f[3.1`]}
ListPlot[Table[{x, f[x]}, {x, -5, 5,  $\frac{1}{2}$ }]]
```

8: Exercise 6 again, using conditional evaluation

Exercise 6 was already written with `/;`, so there is nothing to change. It is repeated here in its Boolean-test form for the record; the two exercises coincide.

```
Clear[f, x];
f[x_] := Sin[x] /; x > 0;
f[x_] := -Sin[x] /; x ≤ 0
Plot[f[x], {x, - $\pi$ ,  $\pi$ }]
```

9: Exercise 3 again, using Which

```
Clear[f, x];
f[x_] := Which[IntegerQ[x], x2, Head[x] === Rational, x3, True, x]
{f[ $\frac{3}{5}$ ], f[3], f[ $\frac{4}{2}$ ], f[3.1`]}
ListPlot[Table[{x, f[x]}, {x, -5, 5,  $\frac{1}{2}$ }]]
```

10: Exercise 6 again, using Which

```
Clear[f, x];
f[x_] := Which[x > 0, Sin[x], True, -Sin[x]]
Plot[f[x], {x, - $\pi$ ,  $\pi$ }]
```

11: Exercise 3 again, as a conditional replacement rule

```
Clear[x, rule3];
rule3 = {n_Integer  $\Rightarrow$  n2, n_Rational  $\Rightarrow$  n3, n_  $\Rightarrow$  n};
{ $\frac{3}{5}$ , 3,  $\frac{4}{2}$ , 3.1`} /. rule3
Replace[{ $\frac{3}{5}$ , 3,  $\frac{4}{2}$ , 3.1`}, rule3, {1}]
ListPlot[Table[{x, Replace[x, rule3]}, {x, -5, 5,  $\frac{1}{2}$ }]]
```

12: Exercise 6 again, as a conditional replacement rule

```
Clear[x, rule6];
rule6 = {n_  $\Rightarrow$  Sin[n] /; n > 0, n_  $\Rightarrow$  -Sin[n]};
{-1, -0.5`, 0, 0.5`, 1} /. rule6
Replace[{-1, -0.5`, 0, 0.5`, 1}, rule6, {1}]
Plot[Replace[x, rule6], {x, - $\pi$ ,  $\pi$ }]
```